splitBarcode software

V2.0.0

User manual

Version	Author	Date	Change
0.1.0	赵福祥	2018.10.22	Support SE/PE, single/dual barcode
0.1.1	赵福祥	2018.10.25	Optimization. Adding -n -m. Compile to static executable. Doesn’t rely on library
0.1.2	赵福祥	2018.11.05	Memory parameter
0.1.3	赵福祥	2018.11.05	Fix BUG: inconsistencies in demultiplex; barcode inconsistencies in SequenceStat
0.1.4	赵福祥	2019.03.22	Fix bug:ambigious reads
0.1.5	赵福祥	2019.04.17	Fix bug:overflow resulted from too many cycles
0.1.6	赵福祥	2019.04.24	Change :-r
1.0.0	唐子淑	2020.03.16	Change format，add parameter for splitting fastq；Support different length for single barcode； Add log file
2.0.0	唐子淑	2020.04.10	Update method to split fastq file

Contents

1. Software Introduction 4

1. Introduction 4

2. Software parameters 5

2 Demonstration 5

1. Data preparation 5

2. User examples 7

1. SE single barcode 7

2. SE dual barcode 7

3. PE single barcode 8

4. PE dual barcode 9

3. Results 10

1. Barcode split file 10

2. Barcode stat file 11

3. FASTQ stat file 12

4. Log file 13

1.Software Introduction

1.Introduction

Software name: splitBarcode

Latest version: V2.0.0

Function: Read specified fastq files and demultiplex based on barcode. Support both SE and PE reads. Support single barcode and dual barcode. Run on both Windows and Linux.

Environment: Windows 10 or Linux (CentOS 7.x). Program is ready to run upon copying.

Help manual: run `splitBarcode` under `bin`, and press enter.

```
$./bin/splitBarcode
USAGE:
  Version 2.0.0,for Basecall Lite:

  cmd [-OPTION]

  if -l,        split fq  files.
  if -F,        split cal file.

if split fq files,      cmd  -B [BarcodeFile] -l [FastqFile] [-OPTION]
  -B STR:               barcode file.[None]
  -l STR:               Fastq for SE mode.[None]
  -2 STR:               Fastq for PE mode.[None]
  -o DIR:               Output dir for decoded fastq.[None]
  -b INT INT INT        Barcode information : startCycle(reads length), lengt
h, mismatchNum.[Last barcode length cycle with 1 mismatch]
  -r:                   Apply reverse complement of barcode sequence.[False]
  -t THREAD:            Set the maximum thread numbers.[CPU number]
  -m MEMORY:            Set the maximum memory(GB).[Available memory]

if split cal files,     cmd  -F [calPath] -C [CycleNum] --Col [ColMax] --Row
[RowMax] [-OPTION]
  -F      STR: the path storing cal files
  -C      INT: the cycle number
  --Col   INT: the max column of fov lists
  --Row   INT: the max row of fov lists
  -P INT: set PE mode, INT is read1Length. Default: SE mode
  -N STR: set slide name. Default: FS2000
  -U INT: set lane number. Default: 1
  -S    : set single fov mode,just one fov
  -E INT: set end cycle process mode. Default: no end cycle process. 1: end c
ycle process for r1, 2: end cycle process for r2, 3: end cycle process for r1
 and r2
  -o DIR: output dir for decoded fastq.[None]
  -B STR: set barcode file. Default: no barcode
  -i INT INT INT        Barcode information : startCycle(real barcode start p
osition), length, mismatchNum.[Last barcode length cycle with 1 mismatch]
  --start_col INT: set start column number. Default: 1
  --start_row INT: set start row number. Default: 1
  -j STR:          set the json file
  --no_filter:     set no filtering,Default:fals
  --filter_para FLOAT[7]: filter parameters information
  --saveNotFiltered:      set save unfiltered reads,Default:not save
  -r:    Apply reverse complement of barcode sequence
  --cpu: Set cpu mode,Default:false
  --umi: Add barcode index into fastq files

[ERROR] : The parameters number is not correct!
```

N.B.

1. Execution permission is required to run this program. In order to change execution permission, run `chmod 755 splitBarcode`

2. Option -F does not apply to BasecallLite. i.e. Litecall doesn't split cal file.

2. Software parameters

Two parameters are required. `-l <fastq file>, -B <barcode file>`, specifying input fastq file and barcode file.

- `-1 <fastq file>` input read 1 fastq file, default to run as SE mode
- `-B <barcode file>` input barcode file. Assuming barcodes are attached at the end. Allowing 1 mismatch.

Optional parameters are as following:

- `-2 <fastq file>` input read 2 fastq file, run in PE mode if this parameter is present.
- `-o <output dir>` output demultiplexed fastq files in designated directory. Default to where fastq files are.
- `-b <readlen barcodelen mis>` Three space-separated parameters required, specifying how barcodes are split. If running with dual barcode, two -b are required. If absent, assume running with single barcode, followed by umber of mismatches allowed.
- `-i <startpos barcodelen mis>` Three space-separated parameters required, specifying how barcodes are split. If running with dual barcode, two -i are required. If absent, assume running with single barcode, followed by umber of mismatches allowed.
- `-t <thread>` Maximum number of threads used. Needs to be an integer greater than 0. E.g. 20.
- `-m <memory>` Maximum amount of memory can be used. Unit is in GB. Needs to be an integer or float greater than 0. E.g. 100.
- `-r` Reverse-complement the barcode sequence. User should choose their method case by case. Default not to run Reverse-complement.

N.B.

If both thread n umber and memory are set, the program can allocate memory by itself. If absent, the program can allocate based on available amount of memory and CPU.

2 Demonstration

1. Data preparation

Assume we have the following:

```
-rw-r--r-- 1 bcbuild ST_BI 1.8K 2020/04/26 15:59 BarcodeV2.1.txt
-rw-r--r-- 1 bcbuild ST_BI  403 2020/04/26 16:13 BarcodeV3.0.txt
drwxr-xr-x 2 bcbuild ST_BI 4.0K 2020/04/26 15:59 bin
drwxr-xr-x 2 bcbuild ST_BI 4.0K 2020/04/26 15:59 lib
-rw-r--r-- 1 bcbuild ST_BI 123M 2020/04/26 16:14 test_read_1.fq.gz
-rw-r--r-- 1 bcbuild ST_BI 141M 2020/04/26 16:14 test_read_2.fq.gz
-rw-r--r-- 1 bcbuild ST_BI 199M 2020/04/26 16:14 test_read.fq.gz
```

- `test_read_1.fq.gz` is read 1 data from PE sequencing. Read length is 100bp.
- `test_read_2.fq.gz` is read 2 data from PE sequencing. Length is 118bp (containing 10bp in barcode).
- `test_read.fq.gz` is read 1 data from SE sequencing. Read length is 100bp (containing 10bp in barcode).
- Two directories `bin` and `lib`. Executable `splitBarcode` and library files are under `bin` and `lib` respectively.
- `BarcodeV2.1.txt` is a tab-delimited text file with two columns. The first and second column contains barcode ID and barcode sequence respectively (10bp in our example).

```
$cat BarcodeV2.1.txt
1       TAGGTCCGAT
2       GGACGGAATC
3       CTTACTGCCG
4       ACCTAATTGA
5       TTCGTATCCG
6       GGTAACGAGC
7       CAACGTATAA
8       ACGTCGCGTT
9       TTCTGCTAGC
10      AGGAAGATAG
11      GCTCTTGCTT
12      CAAGCACGCA
13      CGGCAATCCG
14      ATCAGGATTC
15      TCATTCCAGA
16      GATGCTGGAT
17      GTGAGTGATG
18      GAGTCAGCTG
19      TGTCTGCGAA
```

- `BarcodeV3.0.txt`. Similar to BarcodeV2.1.txt, except the barcodes are 20bp.

```
$cat BarcodeV3.0.txt
9-9     TTCTGCTAGCTTCTGCTAGC
10-10   AGGAAGATAGAGGAAGATAG
11-11   GCTCTTGCTTGCTCTTGCTT
12-12   CAAGCACGCACAAGCACGCA
13-13   CGGCAATCCGCGGCAATCCG
14-14   ATCAGGATTCATCAGGATTC
15-15   TCATTCCAGATCATTCCAGA
16-16   GATGCTGGATGATGCTGGAT
17-17   GTGAGTGATGGTGAGTGATG
18-18   GAGTCAGCTGGAGTCAGCTG
19-19   TGTCTGCGAATGTCTGCGAA
20-20   ATTGGTACAAATTGGTACAA
21-21   CGATTGTGGTCGATTGTGGT
22-22   ACAGACTTCCACAGACTTCC
23-23   TCCACACTCTTCCACACTCT
```

2 User examples

SE single barcode

Assume read 1 is 90bp, with a single 10bp barcode at the end.

run

```
./bin/splitBarcode -B BarcodeV2.1.txt -1 test_read.fq.gz -o
result_se_single_index -b 90 10 1
```

or

```
./bin/splitBarcode -B BarcodeV2.1.txt -1 test_read.fq.gz -o
result_se_single_index -i 91 10 1
```

Details of command:

- `./splitBarcode` the program being used.
- `-B BarcodeV2.1.txt` input barcode file.
- `-1 test_read.fq.gz` read 1 fastq file.
- `-o result_se_single_index` Optional, where output demultiplexed fastq files are directed to.
- `-b 90 10 1` Optional. How barcode is located. Three integers mean read length before barcode, barcode length and barcode mismatch. This example means: there are 90bp of read before barcode, and there is 10bp of barcode, allowing 1 mismatch.
- `-i 91 10 1` Optional. How barcode is located. Three integers mean where the barcode starts, barcode length and barcode mismatch. This example means: barcode begins at 91bp, and there is 10bp of barcode, allowing 1 mismatch.

```
$./bin/splitBarcode  -B BarcodeV2.1.txt -1 test_read.fq.gz -o result_se_singl
e_index -b 90 10 1
Use SE mode
Finish write fastq, total time(s): 23.751
```

Info generated when running. Results are stored in `result_se_single_index` and generated a directory `log` where log files are kept.

```
$ll
total 462M
-rw-r--r-- 1 bcbuild ST_BI 1.8K 2020/04/26 15:59 BarcodeV2.1.txt
-rw-r--r-- 1 bcbuild ST_BI  403 2020/04/26 16:13 BarcodeV3.0.txt
drwxr-xr-x 2 bcbuild ST_BI 4.0K 2020/04/26 15:59 bin
drwxr-xr-x 2 bcbuild ST_BI 4.0K 2020/04/26 15:59 lib
drwxr-xr-x 2 bcbuild ST_BI 4.0K 2020/04/26 16:17 log
drwxr-xr-x 2 bcbuild ST_BI  24K 2020/04/26 16:17 result_se_single_index
-rw-r--r-- 1 bcbuild ST_BI 123M 2020/04/26 16:14 test_read_1.fq.gz
-rw-r--r-- 1 bcbuild ST_BI 141M 2020/04/26 16:14 test_read_2.fq.gz
-rw-r--r-- 1 bcbuild ST_BI 199M 2020/04/26 16:14 test_read.fq.gz
```

SE dual barcode

Assume read 1 is 80bp. Dual barcodes are attached at the end (20bp).

run

```
./bin/splitBarcode -B BarcodeV3.0.txt -1 test_read.fq.gz -o
result_se_double_index -b 80 10 1 -b 90 10 1
```

Or

```
./bin/splitBarcode -B BarcodeV3.0.txt -1 test_read.fq.gz -o
result_se_double_index -i 81 10 1 -i 91 10 1
```

Details of command:

- `./splitBarcode` the program being used.
- `-B BarcodeV3.0.txt` input barcode file.
- `-1 test_read.fq.gz` read 1 fastq file.
- `-o result_se_single_index` Optional, where output demultiplexed fastq files are directed to.
- `-b 80 10 1 -b 90 10 1` Optional. How barcode is located. Three integers mean read length before barcode, barcode length and barcode mismatch. In this dual barcode example: there are 80bp of read before the first barcode, and there is 10bp of barcode, allowing 1 mismatch. Second barcode starts after 90bp, runs 10bp, with 1 mismatch allowed.
- `-i 81 10 1 -i 91 10 1` Optional. How barcode is located. Set of three integers means where the barcode starts, barcode length and barcode mismatch. In this dual barcode example: barcode begins at 81bp, and there is 10bp of barcode, allowing 1 mismatch. Second barcode starts at 91bp, barcode is 10bp and 1 mismatch is allowed.

Info generated when running. Results are stored in `result_se_single_index` and generated a directory log where log files are kept.

```
$./bin/splitBarcode  -B BarcodeV3.0.txt -1 test_read.fq.gz -o result_se_doubl
e_index -b 80 10 1 -b 90 10 1
Use SE mode
Finish write fastq, total time(s): 37.6905
```

PE single barcode

Assume read 1 is 100bp read 2 is 108bp. Single barcode (10bp) located at the end of read 2.

run

```
./bin/splitBarcode -B BarcodeV2.1.txt -1 test_read_1.fq.gz -2
test_read_2.fq.gz -o result_pe_single_index -b 208 10 1 -r
```

or

```
./bin/splitBarcode -B BarcodeV2.1.txt -1 test_read_1.fq.gz -2
test_read_2.fq.gz -o result_pe_single_index -i 209 10 1 -r
```

Details of command:

- `./splitBarcode` the program being used.
- `-B BarcodeV2.1.txt` input barcode file.
- `-1 test_read.fq.gz` read 1 fastq file.
- `-2 test_read.fq.gz` Optional. Read 2 fastq file.
- `-o result_pe_single_index` Optional, specifies where output demultiplexed fastq files are directed to.
- `-b 208 10 1` Optional. How barcode is located. Three integers mean read length before barcode, barcode length and barcode mismatch. This example means: there are 208bp of read before barcode, and there is 10bp of barcode, allowing 1 mismatch.
- `-i 209 10 1` Optional. How barcode is located. Three integers mean where the barcode starts, barcode length and barcode mismatch. This example means:

barcode begins at 209bp, and there is 10bp of barcode, allowing 1 mismatch.

- `-r` Barcode sequences are reverse-complemented. Normally, this option is required when barcode is located at the end of read 2. Needs to be judged on a case-by-case basis.

Info generated when running. Results are stored in `result_pe_single_index`

```
$./bin/splitBarcode -B BarcodeV2.1.txt -1 test_read_1.fq.gz -2 test_read_2.fq
.gz -o result_pe_single_index -b 208 10 1 -r
Use PE mode
Finish write fastq, total time(s): 17.9813
```

PE dual barcode

Assume read 1 is 100bp read 2 is 98bp. Two barcodes (20bp) located at the end of read 2.

run

```
./bin/splitBarcode -B BarcodeV3.0.txt -1 test_read_1.fq.gz -2
test_read_2.fq.gz -o result_pe_double_index -b 198 10 1 -b 208 10 1 - r
```

or

```
./bin/splitBarcode -B BarcodeV3.0.txt -1 test_read_1.fq.gz -2
test_read_2.fq.gz -o result_pe_double_index -i 199 10 1 -i 209 10 1 - r
```

Details of command:

- `./splitBarcode` the program being used.
- `-B BarcodeV3.0.txt` input barcode file.
- `-1 test_read.fq.gz` read 1 fastq file.
- `-2 test_read.fq.gz` Optional. Read 2 fastq file.
- `-o result_pe_single_index` Optional, specifies where output demultiplexed fastq files are directed to.
- `-b 198 10 1 -b 208 10 1` Optional. How barcode is located. Three integers mean read length before barcode, barcode length and barcode mismatch. There are two barcodes in this example: first 198bp are reads, and next 10bp are barcode, allowing 1 mismatch. There are 208bp read prior to second barcode, barcode is 10bp, allowing 1 mismatch.
- `-i 199 10 1 -i 209 10 1` Optional. How barcode is located. Three integers mean where the barcode starts, barcode length and barcode mismatch. There are two barcodes in this example: first barcode begins at 199bp, and there is 10bp of barcode, allowing 1 mismatch. Second barcode begins at 209bp, runs 10bp with 1 mismatch allowed.
- `-r` Barcode sequences are reverse-complemented. Normally, this option is required when barcode is located at the end of read 2. Needs to be judged on a case-by-case basis.

Info generated when running. Results are stored in `result_pe_single_index`

```
$./bin/splitBarcode -B BarcodeV3.0.txt -1 test_read_1.fq.gz -2 test_read_2.fq
.gz -o result_pe_double_index -b 198 10 1 -b 208 10 1 -r
Use PE mode
Finish write fastq, total time(s): 18.2147
```

3. Results

When it finishes running, results can be found under specified directory, which includes fastq files after demultiplexing, fastq stat files and barcode split stat file. Take PE mode single barcode for example. Under directory `result_pe_single_index`, demultiplexed fastq files and barcode stat files can be found.

```
$ll
total 258M
-rw-r--r-- 1 bcbuild ST_BI 1.6K 2020/03/17 11:24 BarcodeStat.txt
-rw-r--r-- 1 bcbuild ST_BI  11K 2020/03/17 11:24 G10000955E320oldDyeB_L01_100_1.fq.fqStat.txt
-rw-r--r-- 1 bcbuild ST_BI  240 2020/03/17 11:24 G10000955E320oldDyeB_L01_100_1.fq.gz
-rw-r--r-- 1 bcbuild ST_BI  12K 2020/03/17 11:24 G10000955E320oldDyeB_L01_100_2.fq.fqStat.txt
-rw-r--r-- 1 bcbuild ST_BI  247 2020/03/17 11:24 G10000955E320oldDyeB_L01_100_2.fq.gz
-rw-r--r-- 1 bcbuild ST_BI  330 2020/03/17 11:24 G10000955E320oldDyeB_L01_101_1.fq.fqStat.txt
-rw-r--r-- 1 bcbuild ST_BI   31 2020/03/17 11:24 G10000955E320oldDyeB_L01_101_1.fq.gz
-rw-r--r-- 1 bcbuild ST_BI  330 2020/03/17 11:24 G10000955E320oldDyeB_L01_101_2.fq.fqStat.txt
-rw-r--r-- 1 bcbuild ST_BI   31 2020/03/17 11:24 G10000955E320oldDyeB_L01_101_2.fq.gz
-rw-r--r-- 1 bcbuild ST_BI  24K 2020/03/17 11:24 G10000955E320oldDyeB_L01_10_1.fq.fqStat.txt
-rw-r--r-- 1 bcbuild ST_BI  33M 2020/03/17 11:24 G10000955E320oldDyeB_L01_10_1.fq.gz
-rw-r--r-- 1 bcbuild ST_BI  330 2020/03/17 11:24 G10000955E320oldDyeB_L01_999_1.fq.fqStat.txt
-rw-r--r-- 1 bcbuild ST_BI   31 2020/03/17 11:24 G10000955E320oldDyeB_L01_999_1.fq.gz
-rw-r--r-- 1 bcbuild ST_BI  330 2020/03/17 11:24 G10000955E320oldDyeB_L01_999_2.fq.fqStat.txt
-rw-r--r-- 1 bcbuild ST_BI   31 2020/03/17 11:24 G10000955E320oldDyeB_L01_999_2.fq.gz
-rw-r--r-- 1 bcbuild ST_BI  23K 2020/03/17 11:24 G10000955E320oldDyeB_L01_undecoded_1.fq.fqStat
.txt
-rw-r--r-- 1 bcbuild ST_BI 8.0M 2020/03/17 11:24 G10000955E320oldDyeB_L01_undecoded_1.fq.gz
-rw-r--r-- 1 bcbuild ST_BI  27K 2020/03/17 11:24 G10000955E320oldDyeB_L01_undecoded_2.fq.fqStat
.txt
-rw-r--r-- 1 bcbuild ST_BI 9.6M 2020/03/17 11:24 G10000955E320oldDyeB_L01_undecoded_2.fq.gz
-rw-r--r-- 1 bcbuild ST_BI 790K 2020/03/17 11:24 SequenceStat.txt
```

1. Barcode split file

File name: `BarcodeStat.txt`
This file contains result of demultiplexing. One key piece of information for assessing sequencing data is the ratio of the number of reads assigned to a barcode to total count, shown in last column in percentage.

Each column represents:

- Barcode ID
- Number of reads assigned to a barcode group without any mismatch
- Number of reads assigned to a barcode group with mismatches detected
- All reads demultiplexed (sum of column 2 and 3)
- Ratio of demultiplexed reads and total reads in percentage.

Last row is the sum for all demultiplexed barcode. The total rate is 93.61%.

N.B.

Ambiguous reads are not included.

```
$cat BarcodeStat.txt
#Barcode        Correct         Corrected       Total   Percentage(%)
barcode3        0       1       1       0.000071
barcode4        1       0       1       0.000071
barcode5        9       5       14      0.000994
barcode6        22      2       24      0.001704
barcode7        13      1       14      0.000994
barcode8        33      2       35      0.002486
barcode9        176714  7690    184404  13.096159
barcode10       356343  18274   374617  26.604866
barcode11       243836  9191    253027  17.969685
barcode12       41836   1721    43557   3.093368
barcode13       63707   3750    67457   4.790718
barcode14       61122   3336    64458   4.577733
barcode15       69034   3662    72696   5.162786
barcode16       41684   1540    43224   3.069719
barcode17       58936   2043    60979   4.330658
barcode18       29773   2802    32575   2.313439
barcode19       25431   1739    27170   1.929582
barcode20       64261   3525    67786   4.814083
barcode21       8593    394     8987    0.638246
barcode22       8428    982     9410    0.668287
barcode23       6977    712     7689    0.546064
barcode28       0       1       1       0.000071
barcode34       0       2       2       0.000142
barcode35       0       1       1       0.000071
barcode36       0       1       1       0.000071
barcode46       0       1       1       0.000071
barcode49       0       1       1       0.000071
barcode52       0       1       1       0.000071
barcode55       0       1       1       0.000071
barcode58       0       1       1       0.000071
barcode65       0       1       1       0.000071
barcode67       0       1       1       0.000071
barcode69       0       1       1       0.000071
barcode70       0       1       1       0.000071
barcode71       0       1       1       0.000071
barcode80       0       1       1       0.000071
barcode87       0       1       1       0.000071
barcode88       0       1       1       0.000071
barcode90       0       2       2       0.000142
barcode96       0       1       1       0.000071
barcode100      0       1       1       0.000071
barcode105      0       1       1       0.000071
barcode107      0       2       2       0.000142
barcode113      0       1       1       0.000071
barcode117      0       8       8       0.000568
barcode124      0       3       3       0.000213
Total   1256753         61409   1318162         93.614342
```

2. Barcode stat file

File name: `SequenceStat.txt`

This file recodes statistics of barcode. If demultiplexing rate is abnormal, this file allows visual examination of distribution of barcode counts. Should incorrect barcodes were used or reverse-complementation parameter was omitted, this file offers diagnostic signs for such mistakes.

Content of four columns are as following:

- Barcode sequence
- Barcode ID
- Count of occurrence
- Percentage in terms of total reads count

```
$cat SequenceStat.txt | head  -20
#Sequence        Barcode        Count    Percentage(%)
CTATCTTCCT       barcode10      356343   25.307068
AAGCAAGAGC       barcode11      243836   17.316951
GCTAGCAGAA       barcode9       176714   12.550024
TCTGGAATGA       barcode15      69034    4.902715
TTGTACCAAT       barcode20      64261    4.563742
CGGATTGCCG       barcode13      63707    4.524397
GAATCCTGAT       barcode14      61122    4.340814
CATCACTCAC       barcode17      58936    4.185567
TGCGTGCTTG       barcode12      41836    2.971144
ATCCAGCATC       barcode16      41684    2.960349
CAGCTGACTC       barcode18      29773    2.114444
TTCGCAGACA       barcode19      25431    1.806080
ACCACAATCG       barcode21      8593     0.610265
GGAAGTCTGT       barcode22      8428     0.598547
AGAGTGTGGA       barcode23      6977     0.495498
CTATATTCCT       barcode10      6332     0.449691
ATATAATAAT       undecoded      3088     0.219306
GCTAGCAGGA       barcode9       2545     0.180743
ATATAATCAT       undecoded      2189     0.155460
```

3. FASTQ stat file

File name: `slide_lane_barcodeID.fq.fqStat.txt`
One `fq.fqStat.txt` file will be generated for each barcode (`slide` and `lane` are placeholders that will be replaced with actual slide ID and lane number). This file contains statistics for demultiplexing, for instance, read count, GC content, Q30 score and base counts.

```
#Name    result_pe_single_index/G10000955E320oldDyeB_L01_undecoded_1.fq.gz
#PhredQual       33
#ReadNum         89915
#row_readLen     100
#col     48
#BaseNum         8991500
#N_Count         2829     0.031463
#GC%     39.40
#Q10%    73.91
#Q20%    58.60
#Q30%    44.49
#EstErr%         7.095463
#Pos     A        C        G        T        N        0        1        2        3        4        5        6
1        27147    23422    14924    24421    1        1        0        0        0        3057     13389    2539
2        37531    15875    20338    16166    5        5        0        0        0        123      1798     2049
3        36439    18157    17582    17730    7        7        0        0        0        237      1994     2118
4        25640    18224    27556    18488    7        7        0        0        0        3741     10989    3497
5        37779    15017    23030    14080    9        9        0        0        0        460      1592     2160
6        11151    27311    14262    37179    12       12       0        0        0        1546     3570     8631
7        16088    23902    15234    34688    3        3        0        0        0        2732     9605     4491
8        34236    21234    18415    16023    7        7        0        0        0        313      1677     3308
9        34231    18586    15368    21724    6        6        0        0        0        190      2730     2403
10       34924    16397    17267    21281    46       46       0        0        0        447      5379     5860
11       36843    16005    17699    19291    77       77       0        0        0        188      1873     2119
12       34914    16450    17442    21095    14       14       0        0        0        2766     2090     2680
13       33579    17411    15654    23257    14       14       0        0        0        218      6252     6618
14       31384    18103    15668    24748    12       12       0        0        0        504      7130     6798
15       33676    17635    15239    23360    5        5        0        0        0        975      4919     11139
```

4. Log file

When this program is running, it generated log files in `log` directory under the current directory. Fastq demultiplexing will generate a plain text file, with name formatted as following: splitBarcode_yyyymmdd_hhmmss-HR.log. One example: splitBarcode_20200317_112434-HR.log. If it uses `cal` file, log file name is as following: cal2fq_yyyymmdd_hhmmss-HR.log. The log file contains program version, parameters, time and such for developing and technical analysis purpose.

N.B.

This program automatically generates a `log` directory if there that directory doesn't exist.